

TEAM_DEVELOPMENT_BREAKDOWN

Team Development Breakdown

HelixCode Feature Implementation Plan

Document Version: 1.0 **Date:** November 7, 2025 **Status:** Implementation Planning

Executive Summary

This document provides a detailed breakdown of how to divide the remaining HelixCode feature implementation across a development team. Based on the AIDER_CLINE_IMPLEMENTATION_ROADMAP.md analysis, we have **16 remaining features** across **4 phases** to implement.

Current Status: - Phase 1 - Critical Features: **2/3 complete** (@ Mentions , Slash Commands , Model Aliases) - Phase 2 - High Priority: **0/4 complete** - Phase 3 - Medium Priority: **0/5 complete** - Phase 4 - Low Priority: **0/4 complete**

Estimated Timeline: 16-20 weeks with a team of 4-6 developers

Team Structure Recommendation

Proposed Team Composition

Team Size: 6 developers + 1 tech lead

Roles: 1. **Tech Lead** (1) - Architecture, code review, integration oversight 2. **Senior Backend Engineers** (2) - Core features, LLM integration, workflow engine 3. **Mid-Level Backend Engineers** (2) - Supporting features, testing, documentation 4. **UI/UX Developer** (1) - TUI enhancements, CLI improvements 5. **QA/Test Engineer** (1) - Test automation, coverage, integration testing

Phase-Based Team Assignment

Phase 1: Critical Features (Weeks 1-4)

Feature 1.3: Model Aliases System

Assigned To: Senior Backend Engineer #1 **Timeline:** Week 1-2 (2 weeks)

Complexity: Medium

Subtasks: 1. **Week 1:** - Design alias configuration schema (YAML/TOML) - Implement alias parser and validator - Create alias resolution engine - Write unit tests for alias matching

1. **Week 2:**

- Integrate with existing LLM provider system
- Add alias autocomplete support
- Create migration tool for existing configs
- Integration tests with all 13 providers

Dependencies: - internal/llm/* (existing provider system) - internal/config/* (configuration management)

Deliverables: - internal/llm/aliases.go - internal/llm/aliases_test.go - config/model-aliases.example.yaml - Documentation: MODEL_ALIASES.md

Acceptance Criteria: - All 13 providers support aliasing - Fuzzy matching with threshold configuration - 100% test coverage - Backwards compatible with existing configs

Phase 2: High Priority Features (Weeks 3-10)

Feature 2.1: Edit Formats (8 formats)

Assigned To: Senior Backend Engineer #2 + Mid-Level Engineer #1 **Timeline:** Week 3-6 (4 weeks) **Complexity:** High

Work Division:

Senior Engineer #2 (Weeks 3-5): - **Week 3:** - Design format abstraction interface - Implement whole-file format (simplest) - Implement diff format with unified diff parser - Unit tests for whole + diff

• **Week 4:**

- Implement udiff format (git-style)
- Implement search/replace format with regex support

- Unit tests for udiff + search/replace
- **Week 5:**
 - Code review and refinement
 - Integration with LLM prompt templates
 - Performance optimization for large files

Mid-Level Engineer #1 (Weeks 4-6): - **Week 4:** - Implement editor format (line-based) - Implement architect mode format - Unit tests for editor + architect

- **Week 5:**
 - Implement ask mode format
 - Implement line-number format
 - Unit tests for ask + line-number
- **Week 6:**
 - Integration testing all 8 formats
 - Format auto-detection logic
 - Documentation and examples

Deliverables: - internal/editor/formats/*.go (8 format implementations) - internal/editor/formats/format_test.go - Documentation: EDIT_FORMATS.md with examples

Dependencies: - internal/editor/* (existing edit infrastructure) - internal/llm/* (for prompt generation)

Acceptance Criteria: - All 8 formats implemented - Auto-detection based on file size/type - Provider-specific format preferences - Diff validation and error recovery - 95%+ test coverage

Feature 2.2: Cline Rules System

Assigned To: Mid-Level Engineer #2 **Timeline:** Week 5-7 (3 weeks) **Complexity:** Medium

Subtasks:

Week 5: - Design rules file format (.clinerules, YAML) - Implement rules parser with validation - Create rule matching engine (glob patterns, regex) - Unit tests for parser and matcher

Week 6: - Integrate with project context system - Implement rule priority/override logic - Add rules to LLM system prompts - Create .clinerules templates for common scenarios

Week 7: - Rules inheritance (workspace → project → file) - Rules editor/validator CLI tool - Integration tests - Documentation with examples

Deliverables: - internal/project/rules.go - internal/project/rules_test.go
- .clinerules.example - cmd/cli/rules.go (rules management commands)

Acceptance Criteria: - Support project, folder, and file-level rules - Pattern matching (glob + regex) - Rule inheritance and override - Integration with @ Mentions and Slash Commands - 90%+ test coverage

Feature 2.3: Focus Chain System

Assigned To: Senior Backend Engineer #1 **Timeline:** Week 7-9 (3 weeks)

Complexity: High

Subtasks:

Week 7: - Design focus state machine (states: planning → building → testing → deploying) - Implement focus context manager - Create state transition logic - Unit tests for state management

Week 8: - Integrate with workflow engine - Implement context preservation across focus changes - Add focus-aware prompt templates - Create focus switch commands

Week 9: - Focus chain visualization (ASCII diagrams) - Focus history tracking and replay - Integration tests with workflows - Documentation

Deliverables: - internal/workflow/focus.go - internal/workflow/focus_test.go - internal/workflow/focus_chain.go - Documentation: FOCUS_CHAIN.md

Acceptance Criteria: - Smooth transitions between focus modes - Context preservation (no information loss) - Integration with existing workflows - Visual feedback in TUI - 95%+ test coverage

Feature 2.4: Hooks System

Assigned To: Mid-Level Engineer #2 **Timeline:** Week 8-10 (3 weeks) **Complexity:** Medium-High

Subtasks:

Week 8: - Design hooks architecture (pre/post event hooks) - Implement hook registry and executor - Create hook context with event data - Unit tests for hook execution

Week 9: - Implement built-in hook events: - pre-commit, post-commit - pre-edit, post-edit - pre-llm-call, post-llm-call - pre-workflow, post-workflow - Add hook configuration (YAML) - Error handling and timeout management

Week 10: - Create example hooks (linting, formatting, notifications) - Security sandboxing for user hooks - Integration tests - Documentation with hook cookbook

Deliverables: - internal/event/hooks.go - internal/event/hooks_test.go - config/hooks.example.yaml - examples/hooks/* (example hooks)

Acceptance Criteria: - Support for 10+ hook events - Async hook execution with timeouts - Hook chaining and dependencies - Security sandboxing - 90%+ test coverage

Phase 3: Medium Priority Features (Weeks 11-18)

Team Assignment for Phase 3:

Parallel Development Strategy:

Track A - Senior Engineer #1 + UI Developer: - Feature 3.1: TUI Improvements (Weeks 11-13) - Feature 3.2: Keyboard Shortcuts (Weeks 14-15)

Track B - Senior Engineer #2 + Mid-Level #1: - Feature 3.3: Provider Settings Management (Weeks 11-13) - Feature 3.4: Session Management (Weeks 14-16)

Track C - Mid-Level #2: - Feature 3.5: File Watcher (Weeks 11-14)

Feature 3.1: TUI Improvements

Assigned To: Senior Backend Engineer #1 (backend) + UI Developer (frontend)

Timeline: Week 11-13 (3 weeks) **Complexity:** Medium-High

Subtasks:

Week 11 (UI Developer): - Design TUI mockups (split panes, color schemes) - Implement chat panel with syntax highlighting - Create file tree navigator component - Implement scrolling and keyboard navigation

Week 11 (Senior Engineer #1): - Refactor TUI architecture (bubble tea framework) - Implement model-view-update pattern - Create component registry - Unit tests for UI logic

Week 12 (Both): - Integrate chat panel with backend - Implement split-pane layouts (vertical/horizontal) - Add terminal output viewer - Create command palette

Week 13 (Both): - Styling and theming system - Color scheme configuration - Responsive layout (terminal resize) - Integration tests

Deliverables: - `cmd/tui/components/.go` - `cmd/tui/layouts/.go` - `config/tui-themes.yaml` - Documentation: `TUI_GUIDE.md`

Acceptance Criteria: - Split panes (chat, files, terminal, problems) - Syntax highlighting for code - Customizable themes - Responsive to terminal resize - Keyboard-driven navigation

Feature 3.2: Keyboard Shortcuts

Assigned To: UI Developer **Timeline:** Week 14-15 (2 weeks) **Complexity:** Medium

Subtasks:

Week 14: - Design keyboard shortcut schema - Implement key binding system - Create default keymaps (vim, emacs, default) - Add shortcut configuration (YAML)

Week 15: - Implement chord support (multi-key sequences) - Add shortcut overlay/help screen - Create keymap editor in TUI - Documentation

Deliverables: - `cmd/tui/keybindings.go` - `config/keybindings.example.yaml` - Documentation: `KEYBOARD_SHORTCUTS.md`

Acceptance Criteria: - Support for 50+ default shortcuts - Customizable keymaps - Vim/Emacs modes - Chord support - Help overlay (? key)

Feature 3.3: Provider Settings Management

Assigned To: Senior Backend Engineer #2 + Mid-Level Engineer #1 **Timeline:** Week 11-13 (3 weeks) **Complexity:** Medium

Subtasks:

Week 11 (Senior #2): - Design provider settings schema - Implement settings storage (encrypted) - Create settings validator - Unit tests

Week 11 (Mid-Level #1): - Implement CLI commands for settings management - Create settings import/export - Add settings templates for common providers

Week 12 (Both): - Integrate settings with LLM providers - Implement runtime settings updates - Add settings inheritance (global → workspace → project)

Week 13 (Both): - Settings migration tool - API key rotation support - Integration tests - Documentation

Deliverables: - internal/config/provider_settings.go - cmd/cli/settings.go - Documentation: PROVIDER_SETTINGS.md

Acceptance Criteria: - Per-provider settings configuration - Encrypted API key storage - Settings validation - Runtime updates (no restart) - 90%+ test coverage

Feature 3.4: Session Management

Assigned To: Senior Backend Engineer #2 + Mid-Level Engineer #1 **Timeline:** Week 14-16 (3 weeks) **Complexity:** Medium-High

Subtasks:

Week 14 (Both): - Design session state schema - Implement session storage (SQLite/PostgreSQL) - Create session lifecycle manager - Unit tests

Week 15 (Senior #2): - Implement session save/restore - Add session search and filtering - Create session branching (fork sessions)

Week 15 (Mid-Level #1): - Implement session export (JSON, markdown) - Create session templates - Add session metadata (tags, descriptions)

Week 16 (Both): - Session compression for long conversations - Session cleanup/archival - Integration tests - Documentation

Deliverables: - internal/session/manager.go - internal/session/storage.go - cmd/cli/session.go - Documentation: SESSION_MANAGEMENT.md

Acceptance Criteria: - Save/restore sessions - Session search and filtering - Session branching/forking - Export to multiple formats - 90%+ test coverage

Feature 3.5: File Watcher

Assigned To: Mid-Level Engineer #2 **Timeline:** Week 11-14 (4 weeks) **Complexity:** Medium

Subtasks:

Week 11: - Research file watcher libraries (fsnotify) - Design watcher architecture
- Implement basic file watching - Unit tests for watcher core

Week 12: - Add debouncing and batching - Implement ignore patterns (.gitignore integration) - Create event filtering - Handle symlinks and mounts

Week 13: - Integrate with @ Mentions (auto-refresh) - Add watcher triggers for workflows - Implement watcher CLI commands

Week 14: - Platform-specific optimizations - Resource usage monitoring -
Integration tests - Documentation

Deliverables: - internal/watcher/watcher.go - internal/watcher/watcher_test.go -
Documentation: FILE_WATCHER.md

Acceptance Criteria: - Real-time file change detection - .gitignore pattern support - Event debouncing (avoid duplicates) - Low resource usage - 85%+ test coverage

Phase 4: Low Priority Features (Weeks 17-20)

Team Assignment for Phase 4:

All Features Parallel: - Feature 4.1: Advanced Diff (Senior #1 - Weeks 17-18) -
Feature 4.2: Streaming Output (Senior #2 - Weeks 17-18) - Feature 4.3: Cost Tracking (Mid-Level #1 - Weeks 17-19) - Feature 4.4: Retry Logic (Mid-Level #2 - Weeks 17-19)

Week 20: Integration testing, bug fixes, documentation cleanup (All hands)

Feature 4.1: Advanced Diff Rendering

Assigned To: Senior Backend Engineer #1 **Timeline:** Week 17-18 (2 weeks)
Complexity: Medium

Subtasks:

Week 17: - Research advanced diff algorithms (Myers, Patience) - Implement syntax-aware diff - Add inline diff highlighting - Unit tests

Week 18: - Integrate with edit formats - Add diff visualization options - Create diff export (HTML, ANSI) - Documentation

Deliverables: - internal/editor/diff/advanced.go - internal/editor/diff/renderer.go

Acceptance Criteria: - Syntax-aware diffing - Multiple diff algorithms -
Beautiful visual output - Export to multiple formats

Feature 4.2: Streaming Output

Assigned To: Senior Backend Engineer #2 **Timeline:** Week 17-18 (2 weeks)
Complexity: Medium

Subtasks:

Week 17: - Implement SSE (Server-Sent Events) for streaming - Add WebSocket support - Create streaming response handler - Unit tests

Week 18: - Integrate with all LLM providers - Add streaming progress indicators - Create cancellation support - Documentation

Deliverables: - internal/llm/streaming.go - internal/server/sse.go

Acceptance Criteria: - Token-by-token streaming - Progress indicators -
Cancellation support - Works with all providers

Feature 4.3: Cost Tracking

Assigned To: Mid-Level Engineer #1 **Timeline:** Week 17-19 (3 weeks) **Complexity:** Medium

Subtasks:

Week 17: - Design cost tracking schema - Implement token counter - Create cost calculator (per-provider pricing) - Unit tests

Week 18: - Add usage dashboard - Implement budget alerts - Create cost reports (daily, weekly, monthly)

Week 19: - Cost optimization suggestions - Integration with all providers - Documentation

Deliverables: - internal/llm/cost_tracker.go - internal/llm/pricing.yaml

Acceptance Criteria: - Accurate token counting - Per-provider cost tracking
- Budget alerts - Usage reports

Feature 4.4: Retry Logic with Backoff

Assigned To: Mid-Level Engineer #2 **Timeline:** Week 17-19 (3 weeks) **Complexity:** Medium

Subtasks:

Week 17: - Design retry strategy (exponential backoff) - Implement retry middleware - Add circuit breaker pattern - Unit tests

Week 18: - Integrate with LLM provider calls - Add configurable retry policies - Implement jitter to prevent thundering herd

Week 19: - Retry observability (metrics, logs) - Integration tests - Documentation

Deliverables: - internal/llm/retry.go - internal/llm/circuit_breaker.go

Acceptance Criteria: - Exponential backoff - Circuit breaker for failing providers - Configurable retry policies - Observability

Testing Strategy

Test Coverage Requirements

Phase 1 (Critical): 95-100% coverage **Phase 2 (High):** 90-95% coverage **Phase 3 (Medium):** 85-90% coverage **Phase 4 (Low):** 80-85% coverage

Test Types by Team Member

QA/Test Engineer (Full-Time): - Integration test suite development - End-to-end test scenarios - Performance/load testing - Test automation (CI/CD integration) - Manual exploratory testing - Regression test maintenance

All Developers: - Unit tests for all new code - Table-driven tests for complex logic - Mock/stub creation for external dependencies - Code coverage reporting

Code Review Process

Review Requirements: 1. All PRs must be reviewed by Tech Lead + 1 peer 2. Critical features (Phase 1-2) require 2 peer reviews 3. Test coverage must meet phase requirements 4. Documentation must be updated 5. Integration tests must pass

Review Timeline: - Small PRs (<200 lines): 24 hours - Medium PRs (200-500 lines): 48 hours - Large PRs (>500 lines): 72 hours (should be split if possible)

Documentation Requirements

Per Feature:

1. **Technical Design Doc** (before implementation)
2. **API Documentation** (godoc comments)
3. **User Guide** (markdown in docs/)
4. **Integration Guide** (for complex features)
5. **Test Plan** (test scenarios and coverage)

Team Documentation:

- **Tech Lead:** Architecture Decision Records (ADRs)
 - **All Developers:** Code comments, README updates
 - **QA:** Test plans, test case documentation
-

Communication & Coordination

Daily Standups (15 min)

- What did you complete yesterday?
- What are you working on today?
- Any blockers or dependencies?

Weekly Planning (1 hour)

- Review completed work
- Plan next week's tasks
- Address technical debt
- Update timeline estimates

Bi-Weekly Architecture Review (2 hours)

- Review ADRs
 - Discuss integration challenges
 - Plan cross-feature coordination
 - Tech debt prioritization
-

Risk Management

Technical Risks

Risk 1: LLM Provider Changes - Mitigation: Provider abstraction layer, versioned APIs - **Owner:** Senior Engineer #2

Risk 2: Performance Issues (File Watcher) - Mitigation: Early performance testing, resource limits - **Owner:** Mid-Level Engineer #2

Risk 3: Integration Complexity (Edit Formats × Providers) - Mitigation: Comprehensive integration test matrix - **Owner:** Senior Engineer #2, QA Engineer

Risk 4: TUI Framework Limitations - Mitigation: POC in Week 1, fallback to simpler UI - **Owner:** UI Developer

Schedule Risks

Risk 1: Underestimated Complexity - Mitigation: 20% buffer time built into estimates - **Owner:** Tech Lead

Risk 2: Key Person Dependency - Mitigation: Knowledge sharing sessions, pair programming - **Owner:** Tech Lead

Success Metrics

Per Phase:

1. **Code Quality:**
 - Test coverage $\geq$ phase requirement
 - Zero critical bugs
 - Code review approval rate $> 95\%$
2. **Timeline:**
 - $\leq 10\%$ variance from estimated timeline
 - No blocking dependencies
3. **Integration:**
 - Seamless integration with existing features
 - No breaking changes to public APIs
 - Backwards compatibility maintained

Overall Project:

- **Completion:** All 16 features implemented
- **Quality:** Average test coverage $\geq 90\%$

- **Performance:** No regression in existing features
 - **Documentation:** 100% feature documentation coverage
-

Handoff & Maintenance

Post-Implementation (Week 21+):

Tech Lead: - Architecture documentation finalization - Technical debt backlog - Performance optimization plan

Senior Engineers: - Knowledge transfer sessions - Advanced feature troubleshooting guide - Performance profiling

Mid-Level Engineers: - User guide finalization - Tutorial videos/walkthroughs - FAQ documentation

QA Engineer: - Regression test suite handoff - Bug tracking and triage process - Performance benchmarks

UI Developer: - Style guide and component library - UI customization guide - Accessibility documentation

Appendix

Technology Stack

Core: - Language: Go 1.24+ - Database: PostgreSQL 14+ (sessions, state) - Cache: Redis 7+ (optional)

TUI: - Framework: bubble tea - Terminal: tcell

Testing: - Unit: testify, gomock - Integration: dockertest - E2E: expect (terminal automation)

CI/CD: - GitHub Actions - Coverage: codecov - Linting: golangci-lint

File Structure

```
helix_code/
├── internal/
│   ├── context/
│   │   └── mentions/           # @ Mentions (    Complete)
│   ├── commands/
│   │   └── builtin/           # Slash Commands (    Complete)
```

└─ custom/	# Custom commands
└─ editor/	
└─ formats/	# Edit Formats (Phase 2)
└─ diff/	# Advanced Diff (Phase 4)
└─ llm/	
└─ aliases.go	# Model Aliases (Phase 1)
└─ cost_tracker.go	# Cost Tracking (Phase 4)
└─ retry.go	# Retry Logic (Phase 4)
└─ streaming.go	# Streaming (Phase 4)
└─ project/	
└─ rules.go	# Cline Rules (Phase 2)
└─ workflow/	
└─ focus.go	# Focus Chain (Phase 2)
└─ focus_chain.go	
└─ event/	
└─ hooks.go	# Hooks System (Phase 2)
└─ session/	
└─ manager.go	# Session Mgmt (Phase 3)
└─ storage.go	
└─ watcher/	
└─ watcher.go	# File Watcher (Phase 3)
└─ config/	
└─ provider_settings.go	# Provider Settings (Phase 3)
└─ cmd/	
└─ tui/	
└─ components/	# TUI Improvements (Phase 3)
└─ layouts/	
└─ keybindings.go	# Keyboard Shortcuts (Phase 3)
└─ cli/	
└─ settings.go	# Settings CLI
└─ session.go	# Session CLI
└─ docs/	
└─ MODEL_ALIASES.md	
└─ EDIT_FORMATS.md	
└─ FOCUS_CHAIN.md	
└─ ... (feature docs)	

Contact & Escalation

Tech Lead: Final authority on architecture decisions **Escalation Path:** Tech Lead
→ Engineering Manager → CTO

Office Hours: - Tech Lead: Daily 2-3 PM for questions - Senior Engineers: On-demand pair programming - QA: Weekly test review sessions (Fridays)

End of Document

Generated by Claude Code (HelixCode) on November 7, 2025